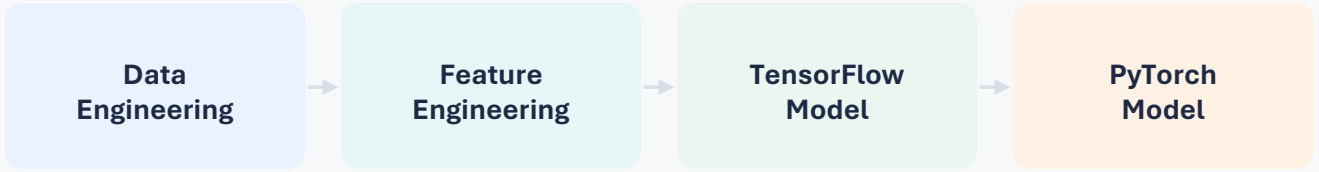
Revenue Forecasting Platform

Using TensorFlow and PyTorch

A comparative machine learning case study built from a data engineering workflow



3,000	80/20	>0.96	2
Dataset rows	Training / testing	Test R^2	Frameworks

Portfolio focus: showing the full path from raw public data to an evaluated business forecast.

Business problem

Turn historical financial and economic data into a defensible revenue forecast.

PROBLEM

Can revenue be forecast using financial drivers and external economic indicators?

- Revenue planning needs more than historical totals.
- The model must use only inputs available before the forecast.
- Model quality should be judged on held-out test data, not just training output.

Project thesis

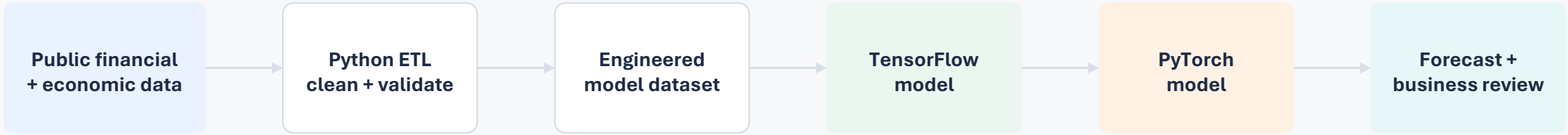
A forecasting platform is strongest when the data pipeline, feature set, model outputs, and performance measures can all be explained clearly.

Business question | Data pipeline | Model review

The goal is not simply to run code. The goal is to tell a credible story from data to forecast.

End-to-end platform view

The deck now tells the full data engineering + machine learning story.

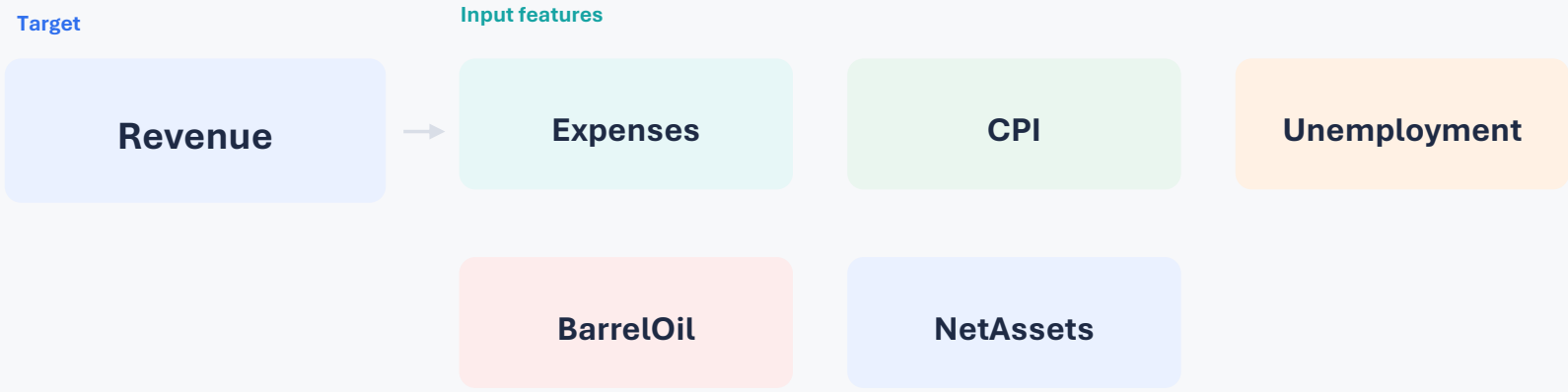


This is the same structure used in migration work: collect, clean, reconcile, validate, test, publish, and explain.

- Data engineering creates the trusted modeling dataset.
- Machine learning is applied only after data quality and leakage checks.
- Evaluation uses test data to confirm generalization.

Data sources and model inputs

The final model used five features to predict Revenue.



Leakage control

- Net Income was removed from the model inputs.
- The 3,000-row no-income dataset keeps the comparison honest.
- Both frameworks trained on the same features and same target.

Why this matters

A clean feature set makes the model review credible and easier to explain to technical or business audiences.

Data engineering pipeline

The workflow aligns with migration and integration roles.

PIPELINE CAPABILITIES

1	**Extract**	Read source files and public indicators
2	**Clean**	Standardize names, numeric types, missing values
3	**Validate**	Row counts, column checks, invalid value checks
4	**Transform**	Build modeling dataset with selected features
5	**Publish**	Export predictions and model summaries

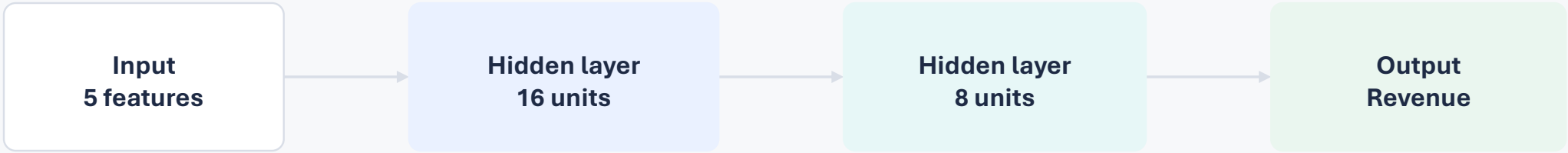
Migration relevance

- Source-to-target thinking
- Data quality controls
- Repeatable Python workflow
- Reconciliation of model inputs and outputs
- Business-friendly validation story

This project can support a migration-focused resume because it shows end-to-end movement from raw inputs to trusted outputs.

Modeling approach

Same forecasting problem, two independent deep learning frameworks.



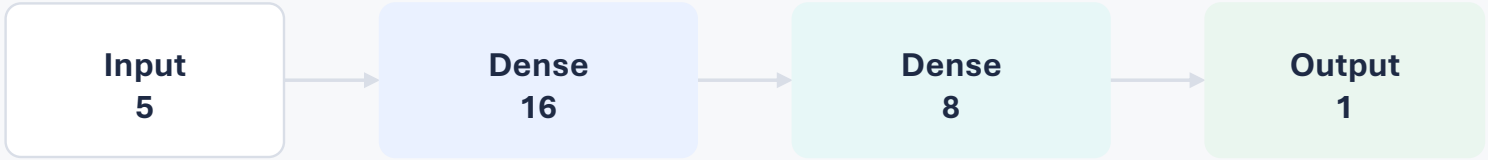
Common configuration

- 3,000-row no-income dataset
- 80/20 train-test split: 2,400 training rows and 600 testing rows
- Adam optimizer and Mean Squared Error loss
- Performance judged using MAE, RMSE, R^2, and training time

TensorFlow implementation

Keras model used the same dataset, feature set, and train-test split.

Architecture



TensorFlow summary

Rows	**3,000**
Train/Test	**2,400 / 600**
Optimizer	**Adam**
Loss	**MSE**
Test R^2	**0.9656**
Training time	**71.3 sec**

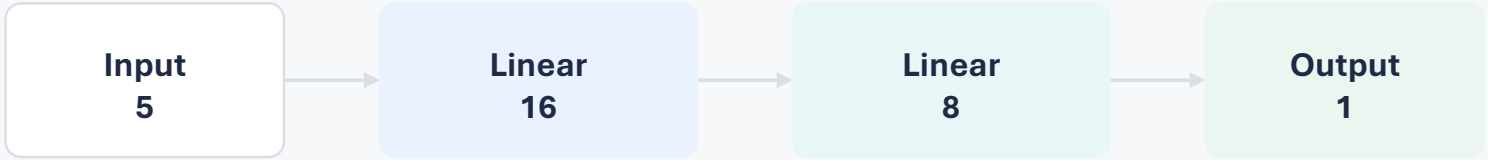
INTERPRETATION

- Strong held-out test performance with R^2 above 0.96.
- Keras API provides clear model structure and training history.
- Useful for presenting model configuration and validation metrics.

PyTorch implementation

Manual training loop configured to parallel the TensorFlow experiment.

Architecture



PyTorch summary

Rows	**3,000**
Train/Test	**2,400 / 600**
Optimizer	**Adam**
Loss	**MSE**
Test R^2	**0.9691**
Training time	**18.9 sec**

INTERPRETATION

- Manual loop made the training process transparent.
- Produced nearly identical test performance compared with TensorFlow.
- Completed training faster in this CPU-based 3,000-row experiment.

TensorFlow vs. PyTorch comparison

The models were configured comparably, making the results easier to interpret.

Metric	PyTorch	TensorFlow
Dataset	3,000 rows	3,000 rows
Features	5	5
Training rows	2,400	2,400
Testing rows	600	600
Optimizer	Adam	Adam
Loss function	MSE	MSE
Hidden layers	16 → 8	16 → 8
Test MAE	$2.07M	$2.19M
Test RMSE	$2.61M	$2.76M
Test R^2	0.9691	0.9656
Training time	18.9 sec	71.3 sec
Future prediction	$68.36M	$58.21M

Headline

The R^2 values are almost identical.

This means both implementations learned essentially the same relationship from the training data and generalized well on the held-out test set.

Not a framework contest — a model evaluation story.

What the results mean

Similar test performance, different training behavior, and different future forecasts.

Test R^2

Training time

Future forecast

Interpretation

- Both models achieved strong predictive performance with R^2 values above 0.96.
- PyTorch completed training about 3.8× faster in this CPU-based experiment.
- The future prediction differed by approximately 15–17%.
- That difference is reasonable because the future row is a new observation, not one of the 600 test rows.

Neither forecast is automatically “correct”; both are plausible outputs from independently trained neural networks.

Key findings

The strongest takeaway is model evaluation, not simply running two tools.

- Identical 3,000-row dataset and five input features were used for both frameworks.
- Both models used the same 80/20 train-test split, Adam optimizer, and MSE loss function.
- Both achieved strong predictive performance with R^2 values above 0.96.
- PyTorch trained approximately 3.8× faster in this CPU-based experiment.
- Future predictions differed by approximately 15–17%, showing that comparable neural networks can produce different yet plausible forecasts.
- Implementing the same business problem in TensorFlow and PyTorch provided practical experience with model configuration, training behavior, and evaluation.

Interview message: I built the pipeline, trained two comparable models, and explained why similar accuracy can still produce different forecasts.

Future work and career bridge

The project supports both ML learning and migration-focused roles.

FUTURE IMPROVEMENTS

- Add more historical years and additional public indicators.
- Use cross-validation to reduce sensitivity to a single train-test split.
- Run hyperparameter tuning and multiple random seeds.
- Test GPU training and larger datasets.
- Automate repeatable model summary exports for reporting.

MIGRATION POSITIONING

- Shows source-to-target data movement and validation.
- Demonstrates ETL/ELT thinking from raw inputs to trusted outputs.
- Uses Python to automate data preparation and model reporting.
- Connects technical outputs to business interpretation.

Career bridge

This portfolio piece supports migration, integration, analytics engineering, and ML data engineering conversations because it proves the full workflow from data to business-ready forecast.